

# Gym Management System - Final Report

**Course:** ICS321

**Submission Date:** May 7, 2026

## **Team Members:**

Jassim AlHumaid

Amjad Alsadiq

Firas Almashama

## Table of Contents

|                                             |    |
|---------------------------------------------|----|
| Gym Management System - Final Report.....   | 1  |
| 1. Problem Statement .....                  | 2  |
| 2. Conceptual Model (ER).....               | 3  |
| 3. Relational Model (Tables) .....          | 5  |
| 4. Creating Tables and Populating Data..... | 7  |
| 5. User Interface Used.....                 | 13 |
| 6. Queries Used in UI.....                  | 13 |
| 7. User Manual.....                         | 16 |
| 8. Conclusion .....                         | 20 |
| 9. Distribution of Work .....               | 21 |

---

## 1. Problem Statement

The day-to-day operations of a modern gym involve many moving parts - registering members, scheduling sessions, managing trainer assignments, handling subscriptions and payments, and tracking attendance. When these activities are managed manually or through disconnected spreadsheets, data quickly becomes inconsistent, redundant, and difficult to maintain. Staff members lose time reconciling records, members face poor service, and management lacks the reliable data needed to make informed decisions.

This project proposes a **Gym Membership and Training Management System**, a centralized data-centric application built on a relational database. The system unifies the gym's core operations - user accounts, members, trainers, plans, subscriptions, payments, sessions, bookings, and attendance - into a single source of truth. Staff and administrators can register members, schedule sessions, approve subscriptions, and track payments through a unified interface, while members and trainers each have their own role-specific views.

The deliverable is a complete full-stack web application with a relational MySQL backend, a TypeScript Express API server, and a React-based front-end with dedicated dashboards for administrators, trainers, and members.

The primary objectives of the Gym Management System are:

- Design a structured relational database that captures all core gym operations without redundancy
- Enforce data consistency and integrity through primary keys, foreign keys, unique constraints, and CHECK constraints
- Provide efficient tracking of members, trainers, sessions, subscriptions, and payments
- Simplify subscription and payment workflows for staff
- Offer role-based access so that members, trainers, and administrators each see only what is relevant to them
- Deliver a scalable foundation that can accommodate future features such as reporting, analytics, and external integrations

The system supports the following functional areas:

- **User management.** A unified user table stores authentication credentials and profile information for all roles. The role of each user (administrator, trainer, or member) is determined by the presence of a row in the corresponding subtype table (staff, trainer, member).
- **Member management.** Member-specific attributes such as gender and join date are recorded, and member profiles are linked to user accounts.
- **Trainer management.** Each trainer is associated with a specialty and a hire date.

- **Membership plans and subscriptions.** Administrators define a catalog of membership plans (monthly, quarterly, annual, student, family, etc.). Members request subscriptions to these plans, and administrators approve or cancel them.
- **Session scheduling.** Trainers schedule sessions with a title, type, date, time range, and capacity. Sessions move through the states *Scheduled*, *Completed*, and *Cancelled*.
- **Booking system.** Members book seats in scheduled sessions, with the database enforcing that no member can book the same session twice.
- **Attendance tracking.** Trainers mark each booking as *Present*, *Absent*, or *Late*. Each booking can have at most one attendance record.
- **Payment management.** Each subscription is linked to a payment record. Administrators review pending payments and mark them as paid or failed.

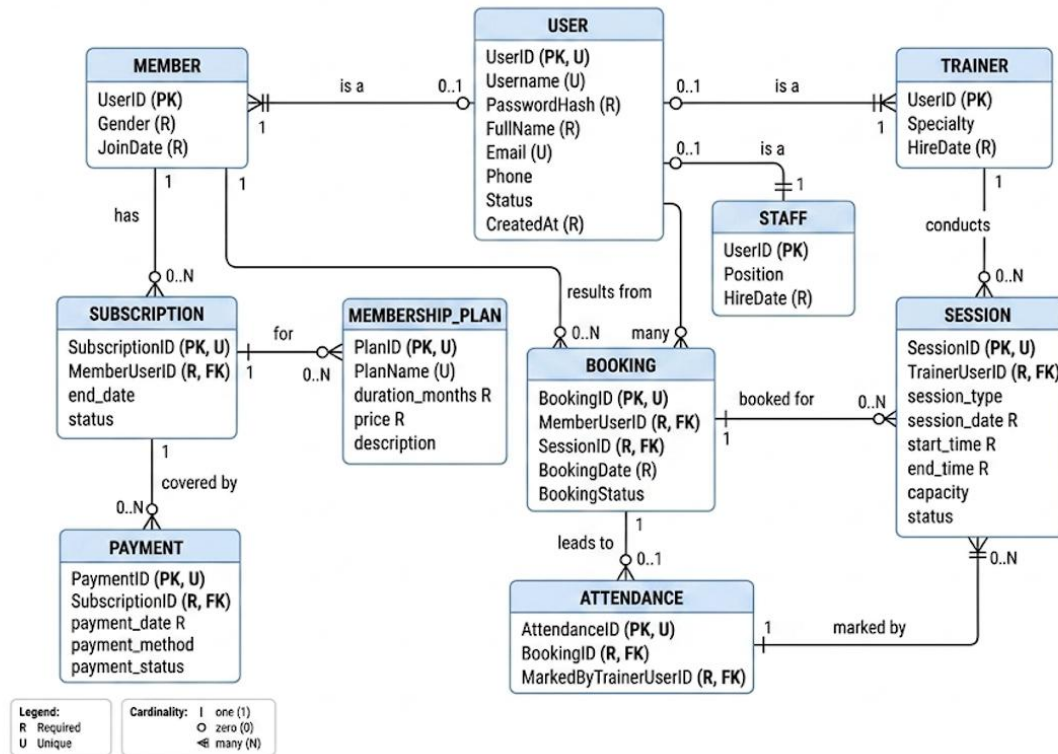
The system does not currently include features such as financial reporting dashboards, equipment inventory, locker assignment, or external payment-gateway integration. These are considered out of scope for the present iteration but are addressed in the *Future Work* section.

---

## 2. Conceptual Model (ER)

The Entity-Relationship (ER) diagram in **Figure 1** shows the entities, attributes, and relationships of the system at the conceptual level.

**Figure 1: Gym Management System - Conceptual ER Diagram**



## Gym Management System - Conceptual ER Diagram

### 2.1 Entities

The system contains ten entities:

- **USER** - Stores authentication credentials and shared profile information for all participants in the system.
- **MEMBER** - A subtype of USER that captures member-specific information such as gender and join date.
- **TRAINER** - A subtype of USER for trainers, with their specialty and hire date.
- **STAFF** - Administrator-specific profile data, including position and hire date.
- **MEMBERSHIPPLAN** - The catalog of subscription plans available for members to purchase.
- **SUBSCRIPTION** - A member's enrollment in a particular plan, with a start date, end date, and lifecycle status.
- **PAYMENT** - A financial transaction associated with a subscription, tracked through statuses Pending, Paid, and Failed.
- **SESSION** - A scheduled training session conducted by a trainer at a given date, time, and capacity.
- **BOOKING** - A reservation by a member for a particular session.
- **ATTENDANCE** - A trainer's record of whether a member who booked a session was present, absent, or late.

## 2.2 Key Relationships

- A USER is a MEMBER, TRAINER, or STAFF (subtype relationship; mutually exclusive in practice).
- A MEMBER **has** zero or many SUBSCRIPTIONs.
- A MEMBERSHIPPLAN **defines** zero or many SUBSCRIPTIONs.
- A SUBSCRIPTION is **covered by** zero or many PAYMENTs.
- A TRAINER **conducts** zero or many SESSIONs.
- A MEMBER **makes** zero or many BOOKINGs, and a SESSION is **booked for** by zero or many BOOKINGs.
- A BOOKING **results in** zero or one ATTENDANCE record (a 1:1 relationship enforced by a UNIQUE constraint on BookingID in the attendance table).
- A TRAINER **marks** zero or many ATTENDANCE records.

## 2.3 Notation

The diagram uses crow's-foot notation for cardinality. Each attribute is annotated with **R** if it is required (NOT NULL) and **U** if it is unique. The bullet (●) marks the primary key.

---

## 3. Relational Model (Tables)

The conceptual ER model from Section 4 maps directly onto a set of relational tables. **Figure 2** shows the relational tables model, where each box represents a database table, PKs are marked with a bullet (●), R indicates required (NOT NULL) columns, U indicates unique columns, and the arrows show foreign key references from the FK table to the referenced PK table.

**Figure 2: Gym Management System - Relational Tables Model**



| Table      | Purpose                                 | Primary Key  | Notable Foreign Keys                                    |
|------------|-----------------------------------------|--------------|---------------------------------------------------------|
| booking    | A member's reservation in a session     | BookingID    | MemberUserID → member,<br>SessionID → session           |
| attendance | Trainer-marked attendance for a booking | AttendanceID | BookingID → booking,<br>MarkedByTrainerUserID → trainer |

### 3.2 Subtype Modeling

The user table holds attributes common to every account: username, password hash, full name, contact details, status, and creation timestamp. Three subtype tables - member, trainer, and staff - each reference UserID as both their primary key and a foreign key into user. This ensures that:

- A subtype row cannot exist without its parent user
- Deleting a user automatically removes the associated subtype row (ON DELETE CASCADE)
- A user's role is determined by which subtype table contains a matching row

This is a cleaner alternative to storing the role as a single column on the user table because it allows each subtype to have its own role-specific attributes (Specialty, HireDate) while keeping the common columns in one place.

### 3.3 Multi-Payment Subscriptions

The payment table allows a single subscription to have multiple payment records at the schema level, although the current application workflow creates a single payment per subscription. The design could support scenarios such as:

- **Installments.** Future work could allow a high-value annual subscription to be paid in installments rather than as one lump sum.
- **Retries.** A follow-up payment record could be inserted after a failed attempt, retaining the history of the original failure for reference.

### 3.4 Booking-Centric Attendance

Attendance is linked to a booking rather than directly to a member and a session. This design choice enforces the business rule that a member can only have attendance recorded for sessions they actually booked. The UNIQUE constraint on BookingID in the attendance table further guarantees that a booking has at most one attendance record.

---

## 4. Creating Tables and Populating Data

The schema applies multiple categories of integrity constraint to enforce data correctness at the database level.

## 4.1 Primary Key Constraints

Every table has a primary key. Surrogate auto-increment integers (UserID, SubscriptionID, etc.) are used wherever a meaningful natural key does not exist.

## 4.2 Unique Constraints

| Table          | Unique Column(s)          | Purpose                                            |
|----------------|---------------------------|----------------------------------------------------|
| user           | Username                  | Each username must be unique system-wide           |
| user           | Email                     | Each email address can be used by at most one user |
| membershipplan | PlanName                  | Plan names must be distinct                        |
| booking        | (MemberUserID, SessionID) | A member cannot book the same session twice        |
| attendance     | BookingID                 | Each booking has at most one attendance record     |

## 4.3 Foreign Key Constraints

Foreign keys enforce referential integrity between tables. The schema uses ON UPDATE CASCADE on every foreign key (so that primary-key changes propagate), and ON DELETE CASCADE on the subtype tables (so that deleting a user automatically removes associated profiles).

## 4.4 CHECK Constraints

CHECK constraints enforce business rules and value-domain restrictions:

| Constraint                  | Rule                                                          |
|-----------------------------|---------------------------------------------------------------|
| chk_user_status             | user.Status must be Active, Inactive, or Suspended            |
| chk_member_gender           | member.Gender must be Male or Female                          |
| chk_subscription_status     | subscription.Status must be Pending, Active, or Cancelled     |
| chk_payment_status          | payment.PaymentStatus must be Pending, Paid, or Failed        |
| chk_session_status          | session.Status must be Scheduled, Completed, or Cancelled     |
| chk_booking_status          | booking.BookingStatus must be Confirmed, Booked, or Cancelled |
| chk_attendance_status       | attendance.AttendanceStatus must be Present, Absent, or Late  |
| chk_membershipplan_duration | membershipplan.DurationMonths must be greater than 0          |



| Constraint               | Rule                                            |
|--------------------------|-------------------------------------------------|
| chk_membershipplan_price | membershipplan.Price must be greater than 0     |
| chk_payment_amount       | payment.Amount must be greater than 0           |
| chk_session_capacity     | session.Capacity must be greater than 0         |
| chk_session_time_order   | session.EndTime must be after session.StartTime |

#### 4.5 NOT NULL Constraints

Every required field is declared NOT NULL, including all primary keys, all foreign keys, and key business attributes such as full name, password hash, plan name, payment amount, and session date and time.

#### 4.6 Database Setup

The schema is defined in `database/schema.sql` and contains all CREATE TABLE statements, primary keys, foreign keys, and CHECK constraints. Sample data is provided in `database/seed.sql`. Both files should be imported after the schema in the correct order (schema first, then seed data).

| Table          | Row Count |
|----------------|-----------|
| user           | 17        |
| staff          | 1         |
| member         | 12        |
| trainer        | 4         |
| membershipplan | 5         |
| subscription   | 14        |
| payment        | 16        |
| session        | 12        |
| booking        | 25        |
| attendance     | 13        |

#### 4.7 Schema Definition

The complete schema is provided in `database/schema.sql`. A condensed listing of each CREATE TABLE statement follows.

```
CREATE DATABASE `gym_management`;
USE `gym_management`;

-- 1. User Table (Base Table)
CREATE TABLE IF NOT EXISTS `user` (
  `UserID` int NOT NULL AUTO_INCREMENT,
```

```

`Username` varchar(50) NOT NULL,
`PasswordHash` varchar(255) NOT NULL,
`FullName` varchar(100) NOT NULL,
`Email` varchar(100) DEFAULT NULL,
`Phone` varchar(20) DEFAULT NULL,
`Status` varchar(20) NOT NULL DEFAULT 'Active',
`CreatedAt` datetime NOT NULL,
PRIMARY KEY (`UserID`),
UNIQUE KEY `Username` (`Username`),
UNIQUE KEY `Email` (`Email`),
CONSTRAINT `chk_user_status` CHECK (`Status` in ('Active',
'Inactive', 'Suspended'))
);

-- 2. Staff Table (UserID is the Primary Key)
CREATE TABLE IF NOT EXISTS `staff` (
  `UserID` int NOT NULL,
  `Position` varchar(100) DEFAULT 'Admin',
  `HireDate` date NOT NULL,
  PRIMARY KEY (`UserID`),
  CONSTRAINT `fk_staff_user` FOREIGN KEY (`UserID`) REFERENCES `user`
  (`UserID`) ON DELETE CASCADE ON UPDATE CASCADE
);

-- 3. Trainer Table (UserID is the Primary Key)
CREATE TABLE IF NOT EXISTS `trainer` (
  `UserID` int NOT NULL,
  `Specialty` varchar(100) DEFAULT NULL,
  `HireDate` date NOT NULL,
  PRIMARY KEY (`UserID`),
  CONSTRAINT `fk_trainer_user` FOREIGN KEY (`UserID`) REFERENCES
  `user` (`UserID`) ON DELETE CASCADE ON UPDATE CASCADE
);

-- 4. Member Table (UserID is the Primary Key)
CREATE TABLE IF NOT EXISTS `member` (
  `UserID` int NOT NULL,
  `Gender` varchar(20) NOT NULL,
  `JoinDate` date NOT NULL,
  PRIMARY KEY (`UserID`),
  CONSTRAINT `fk_member_user` FOREIGN KEY (`UserID`) REFERENCES `user`
  (`UserID`) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `chk_member_gender` CHECK (`Gender` in ('Male',
'Female'))
);

-- 5. Membership Plans
CREATE TABLE IF NOT EXISTS `membershipplan` (
  `PlanID` int NOT NULL AUTO_INCREMENT,
  `PlanName` varchar(100) NOT NULL,
  `DurationMonths` int NOT NULL,
  `Price` decimal(10, 2) NOT NULL,

```

```

    `Description` varchar(255) DEFAULT NULL,
    PRIMARY KEY (`PlanID`),
    UNIQUE KEY `PlanName` (`PlanName`),
    CONSTRAINT `chk_membershipplan_duration` CHECK (`DurationMonths` >
0),
    CONSTRAINT `chk_membershipplan_price` CHECK (`Price` > 0)
);

-- 6. Subscriptions
CREATE TABLE IF NOT EXISTS `subscription` (
    `SubscriptionID` int NOT NULL AUTO_INCREMENT,
    `MemberUserID` int NOT NULL,
    `PlanID` int NOT NULL,
    `StartDate` date NOT NULL,
    `EndDate` date DEFAULT NULL,
    `Status` varchar(20) NOT NULL DEFAULT 'Pending',
    PRIMARY KEY (`SubscriptionID`),
    CONSTRAINT `fk_subscription_member` FOREIGN KEY (`MemberUserID`)
REFERENCES `member` (`UserID`) ON UPDATE CASCADE,
    CONSTRAINT `fk_subscription_plan` FOREIGN KEY (`PlanID`) REFERENCES
`membershipplan` (`PlanID`) ON UPDATE CASCADE,
    CONSTRAINT `chk_subscription_status` CHECK (`Status` in ('Pending',
'Active', 'Cancelled'))
);

-- 7. Payments
CREATE TABLE IF NOT EXISTS `payment` (
    `PaymentID` int NOT NULL AUTO_INCREMENT,
    `SubscriptionID` int NOT NULL,
    `Amount` decimal(10, 2) NOT NULL,
    `PaymentDate` date NOT NULL,
    `PaymentMethod` varchar(50) DEFAULT NULL,
    `PaymentStatus` varchar(20) NOT NULL DEFAULT 'Pending',
    PRIMARY KEY (`PaymentID`),
    CONSTRAINT `fk_payment_subscription` FOREIGN KEY (`SubscriptionID`)
REFERENCES `subscription` (`SubscriptionID`) ON UPDATE CASCADE,
    CONSTRAINT `chk_payment_amount` CHECK (`Amount` > 0),
    CONSTRAINT `chk_payment_status` CHECK (`PaymentStatus` in
('Pending', 'Paid', 'Failed'))
);

-- 8. Training Sessions
CREATE TABLE IF NOT EXISTS `session` (
    `SessionID` int NOT NULL AUTO_INCREMENT,
    `TrainerUserID` int NOT NULL,
    `SessionTitle` varchar(100) NOT NULL,
    `SessionType` varchar(50) DEFAULT NULL,
    `SessionDate` date NOT NULL,
    `StartTime` time NOT NULL,
    `EndTime` time NOT NULL,
    `Capacity` int DEFAULT NULL,
    `Status` varchar(20) NOT NULL DEFAULT 'Scheduled',

```

```

    PRIMARY KEY (`SessionID`),
    CONSTRAINT `fk_session_trainer` FOREIGN KEY (`TrainerUserID`)
REFERENCES `trainer` (`UserID`) ON UPDATE CASCADE,
    CONSTRAINT `chk_session_capacity` CHECK (`Capacity` > 0),
    CONSTRAINT `chk_session_status` CHECK (`Status` in ('Scheduled',
'Completed', 'Cancelled')),
    CONSTRAINT `chk_session_time_order` CHECK (`EndTime` > `StartTime`)
);

```

-- 9. Bookings

```

CREATE TABLE IF NOT EXISTS `booking` (
    `BookingID` int NOT NULL AUTO_INCREMENT,
    `MemberUserID` int NOT NULL,
    `SessionID` int NOT NULL,
    `BookingDate` date NOT NULL,
    `BookingStatus` varchar(20) NOT NULL DEFAULT 'Confirmed',
    PRIMARY KEY (`BookingID`),
    UNIQUE KEY `uq_booking_member_session` (`MemberUserID`,
`SessionID`),
    CONSTRAINT `fk_booking_member` FOREIGN KEY (`MemberUserID`)
REFERENCES `member` (`UserID`) ON UPDATE CASCADE,
    CONSTRAINT `fk_booking_session` FOREIGN KEY (`SessionID`) REFERENCES
`session` (`SessionID`) ON UPDATE CASCADE,
    CONSTRAINT `chk_booking_status` CHECK (`BookingStatus` in
('Confirmed', 'Booked', 'Cancelled'))
);

```

-- 10. Attendance

```

CREATE TABLE IF NOT EXISTS `attendance` (
    `AttendanceID` int NOT NULL AUTO_INCREMENT,
    `BookingID` int NOT NULL,
    `MarkedByTrainerUserID` int NOT NULL,
    `AttendanceStatus` varchar(20) NOT NULL,
    `MarkedAt` datetime DEFAULT NULL,
    PRIMARY KEY (`AttendanceID`),
    UNIQUE KEY `uq_attendance_booking` (`BookingID`),
    CONSTRAINT `fk_attendance_booking` FOREIGN KEY (`BookingID`)
REFERENCES `booking` (`BookingID`) ON UPDATE CASCADE,
    CONSTRAINT `fk_attendance_trainer` FOREIGN KEY
(`MarkedByTrainerUserID`) REFERENCES `trainer` (`UserID`) ON UPDATE
CASCADE,
    CONSTRAINT `chk_attendance_status` CHECK (`AttendanceStatus` in
('Present', 'Absent', 'Late'))
);

```

## 5. User Interface Used

### 5.1 Technology Stack

| Layer          | Technology                                        |
|----------------|---------------------------------------------------|
| Database       | MySQL (mysql2 driver)                             |
| Backend        | Node.js, Express 4, TypeScript, tsx               |
| Authentication | JSON Web Tokens (JWT), bcrypt password hashing    |
| Frontend       | React 19, TypeScript, Vite 6, React Router 7      |
| Styling        | Custom CSS                                        |
| Testing        | Vitest + Supertest — 69 tests across 7 test files |

### 5.2 Frontend Architecture

The React frontend has three role-specific dashboards:

- **AdminDashboard** - system-wide metrics, plan management, member and trainer listings, subscription approvals, payment overview, and session management.
- **TrainerDashboard** - sessions conducted by the logged-in trainer and an attendance-marking interface for each session.
- **MemberDashboard** - the member's subscriptions, payments, available sessions to book, and existing bookings.

A shared LoginPage and LandingPage handle unauthenticated users, with React Router redirecting users to the correct dashboard based on their role after login.

## 6. Queries Used in UI

### 6.1 API Endpoints

The Express server exposes a REST API under the /api prefix. Routes are organized by resource:

- /api/auth - login and registration
- /api/dashboard - admin metrics
- /api/users, /api/members, /api/trainers - user-management endpoints
- /api/plans - membership plan CRUD
- /api/subscriptions - subscription request, approval, and cancellation
- /api/payments - payment listing
- /api/sessions - session scheduling
- /api/bookings - booking creation and cancellation
- /api/attendance - attendance marking by trainers

Authentication is handled by issuing a signed JWT on successful login. The JWT carries the user's role (derived from which subtype table contains the matching row), and route middleware enforces role-based access control.

---

This section presents representative SQL queries that power the application. Each query is annotated with its purpose.

## 6.2 Login (Role-Aware)

This query fetches a user by username and derives the role from the presence of rows in the subtype tables. The `LEFT JOIN` to all three subtype tables allows a single round-trip to determine identity and role.

```
SELECT u.UserID AS user_id,
       u.Username AS username,
       u.PasswordHash AS password_hash,
       u.FullName AS full_name,
       u.Email AS email,
       LOWER(u.Status) AS status,
       CASE
         WHEN st.UserID IS NOT NULL THEN 'admin'
         WHEN t.UserID IS NOT NULL THEN 'trainer'
         WHEN m.UserID IS NOT NULL THEN 'member'
       END AS role,
       m.UserID AS member_id,
       t.UserID AS trainer_id
FROM   `user` u
LEFT JOIN staff st ON st.UserID = u.UserID
LEFT JOIN trainer t ON t.UserID = u.UserID
LEFT JOIN member m ON m.UserID = u.UserID
WHERE  u.Username = :username
LIMIT 1
```

## 6.3 Admin Dashboard Counts

The administrator dashboard issues a series of `COUNT(*)` queries to populate top-level metrics. A representative subset:

```
SELECT COUNT(*) AS count
FROM   `user` u
JOIN   member m ON m.UserID = u.UserID
WHERE  u.Status = 'Active';

SELECT COUNT(*) AS count
FROM   subscription
WHERE  Status = 'Active';

SELECT COUNT(*) AS count
FROM   `session`
WHERE  Status = 'Scheduled';
```

## 6.4 Subscription Workflow - Request, Approve, Cancel

A member requests a new subscription:

```
INSERT INTO subscription (MemberUserID, PlanID, StartDate, EndDate, Status)
VALUES (:member_id, :plan_id, :start_date, :end_date, 'Pending');
```

When an administrator approves a pending subscription, the application creates a pending payment record for the plan price. The subscription remains Pending until an administrator marks that payment Paid, at which point the subscription activates and any other active subscriptions for the same member are cancelled:

```
-- Admin approves: creates a pending payment
INSERT INTO payment (SubscriptionID, Amount, PaymentDate, PaymentMethod, PaymentStatus)
VALUES (:subscription_id, :amount, CURDATE(), NULL, 'Pending');

-- Admin marks payment paid: cancels other active subs
UPDATE subscription
  SET Status = 'Cancelled'
  WHERE MemberUserID = :member_id AND SubscriptionID <> :subscription_id AND Status = 'Active';

-- Admin marks payment paid: activates the pending sub
UPDATE subscription
  SET Status = 'Active'
  WHERE SubscriptionID = :subscription_id AND Status = 'Pending';

UPDATE subscription SET Status = 'Cancelled' WHERE SubscriptionID = :subscription_id
```

An administrator can cancel a subscription directly:

## 6.5 Sessions With Booked-Count

The session listing includes a derived `booked_count` column showing how many active bookings each session has. This uses a correlated subquery with a `LEFT JOIN`. For members, the listing is filtered to only scheduled sessions; trainers see only their own sessions; administrators see all sessions.

```
SELECT s.SessionID AS session_id,
       s.TrainerUserID AS trainer_id,
       s.SessionTitle AS session_title,
       s.SessionType AS session_type,
       s.SessionDate AS session_date,
       s.StartTime AS start_time,
       s.EndTime AS end_time,
       s.Capacity AS capacity,
       LOWER(s.Status) AS status,
       u.FullName AS trainer_name,
       t.Specialty AS trainer_specialty,
       COALESCE(booked.booked_count, 0) AS booked_count
FROM `session` s
JOIN trainer t ON t.UserID = s.TrainerUserID
JOIN `user` u ON u.UserID = t.UserID
LEFT JOIN (
  SELECT SessionID, COUNT(*) AS booked_count
  FROM booking
  WHERE BookingStatus IN ('Confirmed', 'Booked')
  GROUP BY SessionID
) booked ON booked.SessionID = s.SessionID
WHERE s.Status = 'Scheduled'
ORDER BY s.SessionDate, s.StartTime
```

## 6.6 Trainer Attendance View

For each session a trainer is conducting, this query lists the booked members and any existing attendance record:

```

SELECT b.MemberUserID AS member_id,
       u.FullName AS member_name,
       u.Email AS member_email,
       a.AttendanceID AS attendance_id,
       LOWER(a.AttendanceStatus) AS attendance_status,
       a.MarkedByTrainerUserID AS marked_by_trainer_id,
       a.MarkedAt AS marked_at
FROM booking b
JOIN member m ON m.UserID = b.MemberUserID
JOIN `user` u ON u.UserID = m.UserID
LEFT JOIN attendance a ON a.BookingID = b.BookingID
WHERE b.SessionID = :session_id AND b.BookingStatus IN ('Confirmed', 'Booked')
ORDER BY u.FullName

```

## 6.7 Marking Attendance (Idempotent)

The attendance-marking endpoint uses an INSERT ... ON DUPLICATE KEY UPDATE pattern to support both initial marking and subsequent corrections:

```

INSERT INTO attendance (BookingID, MarkedByTrainerUserID, AttendanceStatus,
MarkedAt)
VALUES (:booking_id, :trainer_id, :attendance_status, NOW())
ON DUPLICATE KEY UPDATE
    MarkedByTrainerUserID = VALUES(MarkedByTrainerUserID),
    AttendanceStatus      = VALUES(AttendanceStatus),
    MarkedAt              = NOW();

```

The UNIQUE constraint on BookingID makes this safe and predictable.

## 7. User Manual

The application provides three role-specific dashboards. Screenshots of each are shown below.





GYM MANAGEMENT SYSTEM

PULSE STUDIO ACCESS

# WELCOME BACK

Use the seeded admin profile or register a new member account. The demo login uses `password123`.

Quick sample login

Admin User - admin



Login

Register

Username

admin1

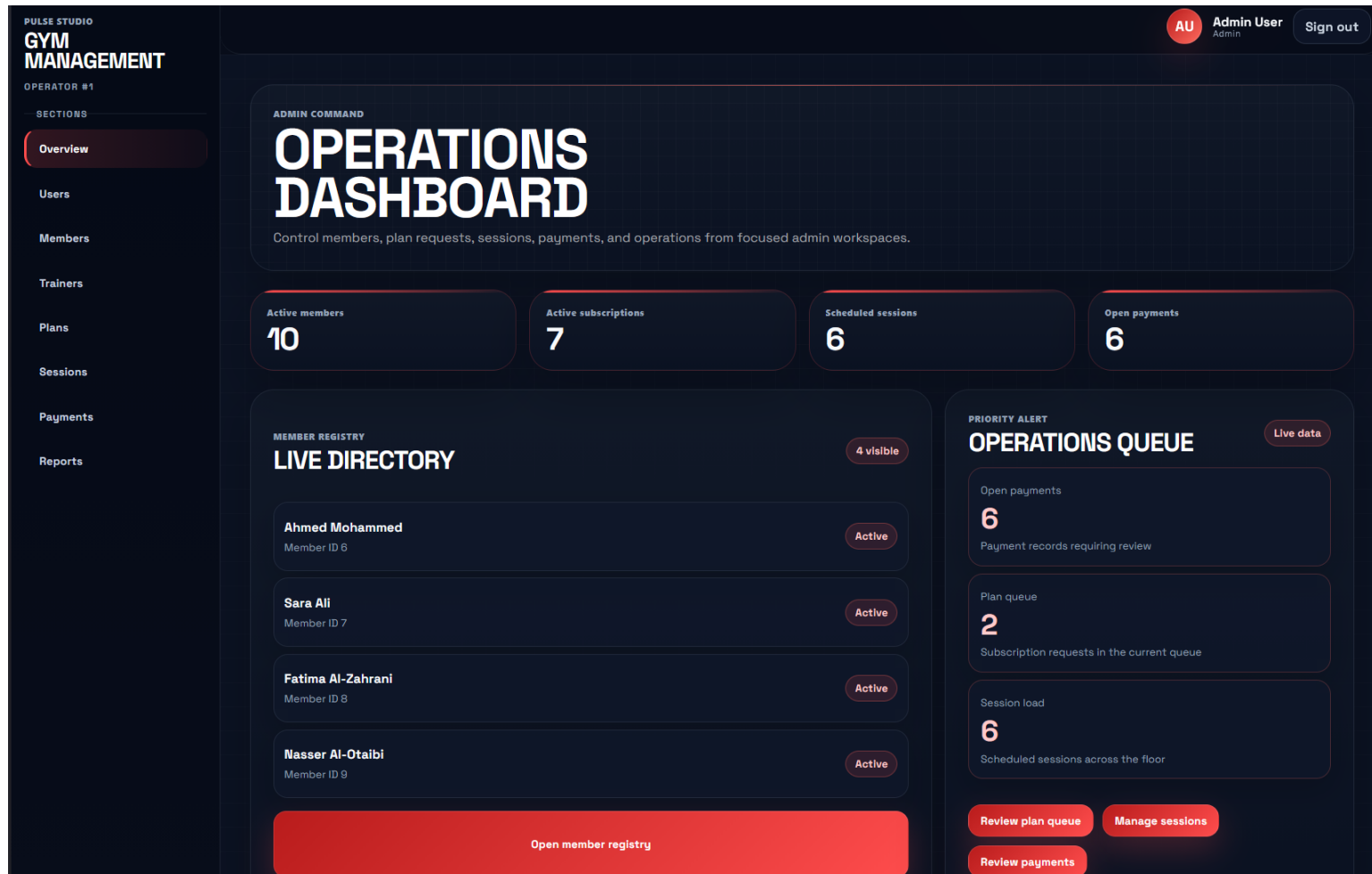
Password

••••••••

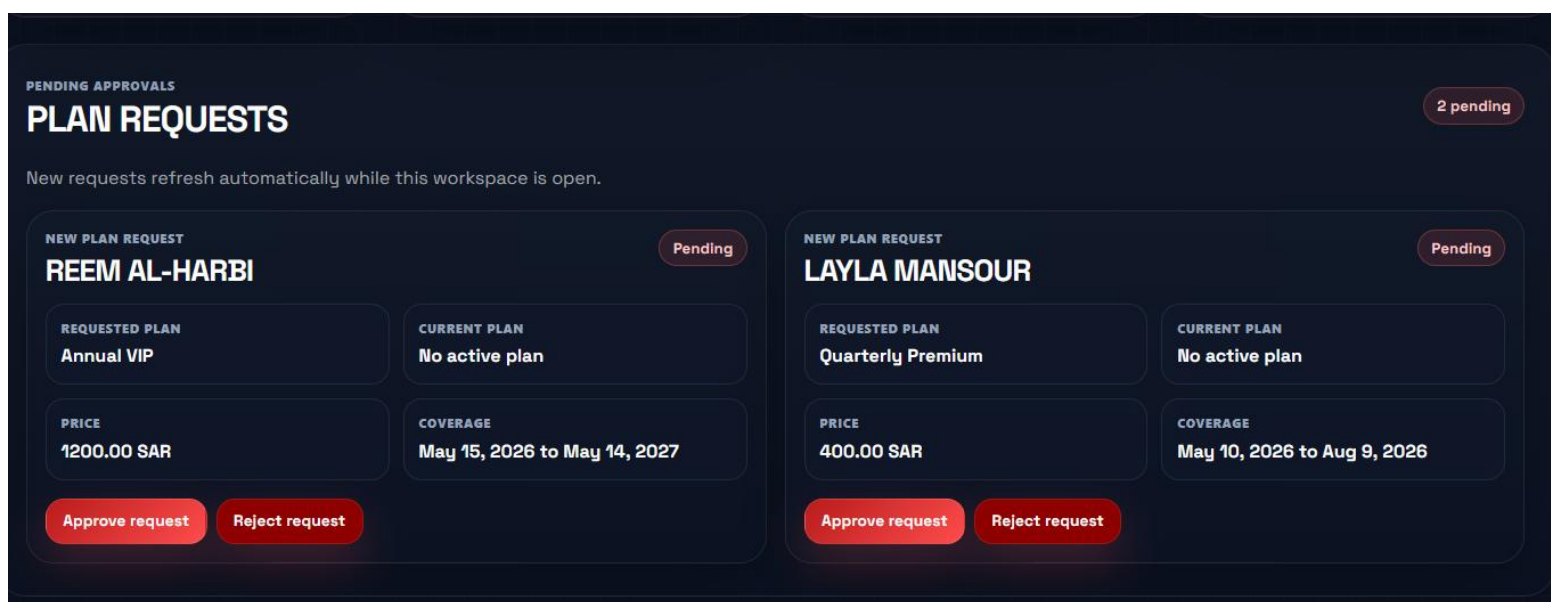
Sign in

[Back to landing page](#)

Login page



Admin dashboard



Admin subscription approval view

PULSE STUDIO

GYM MANAGEMENT

OPERATOR #2

SECTIONS

Upcoming

Completed

Mark attendance

History

KA

Khalid Al-Saadi

Trainer

Sign out

TRAINER FLOOR

SESSIONS AND ATTENDANCE CONTROL

Review assigned sessions, load booked members, and submit attendance without leaving your role workspace.

ACTIVE PROTOCOLS

2

Scheduled sessions assigned to your trainer profile.

ATTENDANCE QUEUE

3

Booked members loaded for the selected session.

SELECTED SESSION

STRENGTH

Apr 10, 2026 at 7:00 AM

2 completed

RESOURCE

UPCOMING ASSIGNED SESSIONS

2 rows

| ID | Type     | Date         | Start   | End     | Booked | Capacity | Status    |
|----|----------|--------------|---------|---------|--------|----------|-----------|
| 6  | Strength | May 8, 2026  | 7:00 AM | 8:00 AM | 2      | 12       | scheduled |
| 7  | Strength | May 15, 2026 | 6:00 PM | 7:30 PM | 1      | 8        | scheduled |

Trainer dashboard

PULSE STUDIO

GYM MANAGEMENT

OPERATOR #6

SECTIONS

Overview

Plans

Sessions

Bookings

AM

Ahmed Mohammed

Member

Sign out

MEMBER HUB

MEMBERSHIP AND BOOKING CONTROL

Review your plan status, request a subscription, book sessions, and track payments.

CURRENT ACTIVE PLAN

ANNUAL VIP

Feb 5, 2026 to Feb 4, 2027

active

PENDING REQUEST

NO PENDING REQUEST

New requests will appear here until admin reviews them.

Clear

LATEST PAYMENT

600.00 SAR

Annual VIP on Aug 5, 2026

pending

SESSION ACCESS

4 BOOKED

Next available: Strength with Khalid Al-Saadi.

Booking enabled

Member dashboard

| payment id | member name       | plan name         | amount  | payment date | payment method | payment status | Actions   |             |
|------------|-------------------|-------------------|---------|--------------|----------------|----------------|-----------|-------------|
| 3          | Ahmed Mohammed    | Annual VIP        | 600.00  | Aug 5, 2026  | Card           | pending        | Mark paid | Mark failed |
| 15         | Reem Al-Harbi     | Annual VIP        | 1200.00 | May 12, 2026 | Card           | pending        | Mark paid | Mark failed |
| 11         | Layla Mansour     | Quarterly Premium | 400.00  | May 8, 2026  | Transfer       | pending        | Mark paid | Mark failed |
| 16         | Majed Al-Najdi    | Monthly Basic     | 150.00  | Apr 15, 2026 | Card           | failed         | Mark paid | Mark failed |
| 9          | Huda Al-Shehri    | Monthly Basic     | 150.00  | Apr 1, 2026  | Card           | paid           | Mark paid | Mark failed |
| 14         | Omar Al-Qahtani   | Monthly Basic     | 150.00  | Apr 1, 2026  | Cash           | paid           | Mark paid | Mark failed |
| 13         | Aisha Al-Balushi  | Student Monthly   | 100.00  | Mar 15, 2026 | Card           | paid           | Mark paid | Mark failed |
| 12         | Tariq Al-Ahmadi   | Monthly Basic     | 150.00  | Mar 10, 2026 | Card           | failed         | Mark paid | Mark failed |
| 10         | Ibrahim Al-Rashid | Family Quarterly  | 650.00  | Feb 20, 2026 | Transfer       | paid           | Mark paid | Mark failed |
| 8          | Huda Al-Shehri    | Monthly Basic     | 150.00  | Feb 10, 2026 | Card           | paid           | Mark paid | Mark failed |

## Payment management

Each interface respects the user's role: members can only view and act on their own data, trainers can only mark attendance for sessions they conduct, and administrators have full visibility and management privileges.

## 8. Conclusion

This project delivers a complete, working Gym Management System backed by a normalized relational database. The schema cleanly captures the core gym operations through ten interrelated tables and applies a comprehensive set of integrity constraints - primary keys, unique keys, foreign keys, and CHECK constraints - to ensure that the data remains consistent regardless of how it is accessed.

The system demonstrates a number of non-trivial design decisions: subtype tables for distinguishing roles, schema-level multi-payment subscription design, booking-centric attendance for enforcing the booking-attendance relationship, and a CASE-based role derivation in the login query that avoids storing redundant role data.

### 8.1 Future Work

Possible extensions to the system include:

- **Reporting and analytics.** Dashboards showing revenue trends, attendance rates, and trainer utilization.
- **Equipment and locker management.** Additional tables for gym equipment inventory and locker assignments.

- **External payment integration.** Linking the payment table to a real payment-gateway API such as Stripe or PayTabs.
- **Notifications.** Email or SMS reminders for upcoming sessions and expiring subscriptions.
- **Mobile application.** A native iOS/Android client that consumes the same REST API.
- **Audit logging.** A separate `audit_log` table tracking all sensitive operations (subscription approvals, payment status changes, attendance marking) for compliance.

## 8.2 Lessons Learned

Building this system required careful coordination between the conceptual ER model, the relational schema, and the application code. Small inconsistencies between layers - a column missing from the ER diagram, a CHECK constraint not reflected in the application's input validation - could have caused integration friction late in the project. The team's approach of treating the schema as the source of truth, and regenerating the ER diagram and front-end forms from it, kept the three layers aligned throughout development.

## 9. Distribution of Work

**Conceptual Model (ER):** All members contributed equally to designing entities, attributes, and relationships.

**Relational Model:** All members contributed equally to mapping the ER diagram into normalized tables, defining keys, and specifying constraints.

**Creating Tables and Populating Data:** All members participated in writing the schema and seeding the database with sample data.

**Frontend and User Interface:** Jassim took the lead on implementing the React frontend, designing role-specific dashboards, and compiling the user manual with screenshots.

**Backend and API:** Amjad and Firas focused on building the Express API endpoints, implementing business logic for subscriptions and payments, writing the SQL queries used in the UI, and documenting backend architecture.

---